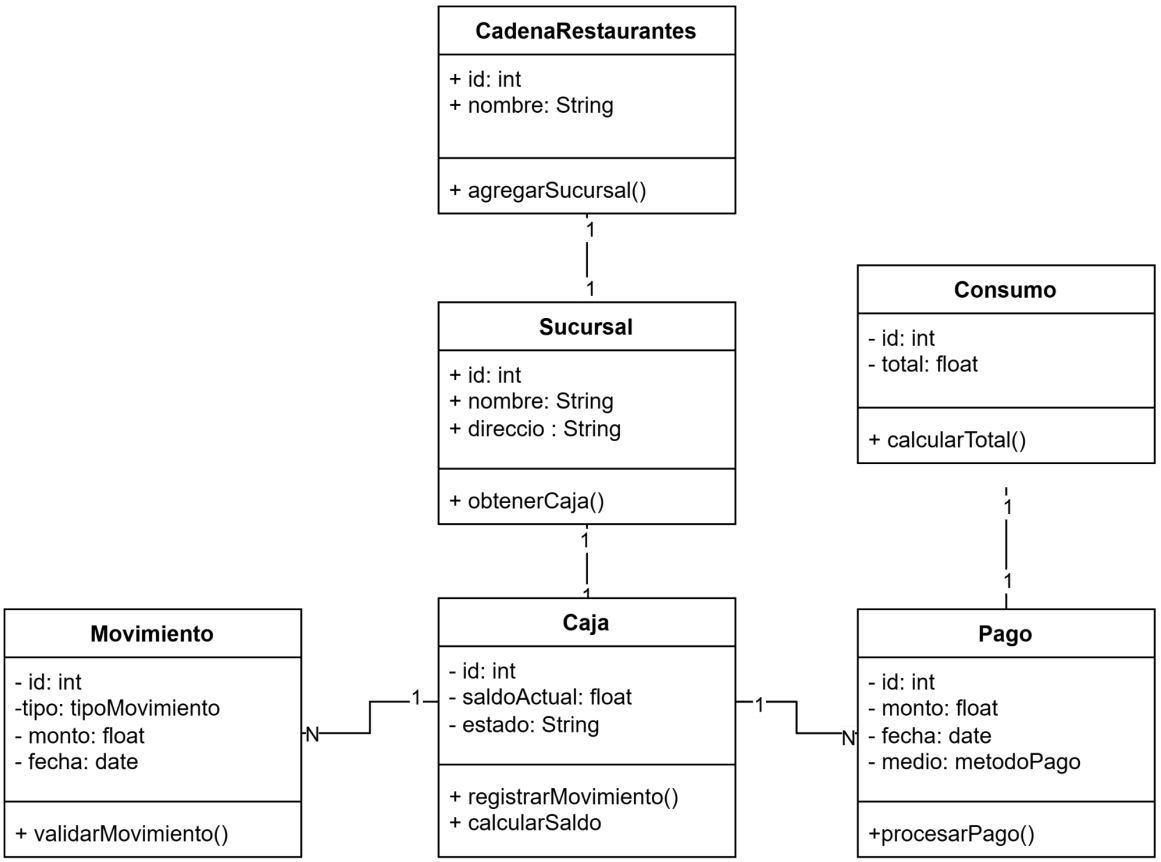
Ejercicio N°6: INTEGRACIÓN CONTINUA Y DESPLIEGUE.

a) Desarrollar el Diagrama de Clases de Diseño con sus atributos, relaciones, cardinalidades y métodos correspondiente del siguiente problema planteado: El equipo trabaja en el desarrollo de software para una cadena de restaurante (sucursal) donde cada restaurant posee una caja propia donde se registran los movimientos de ingreso y salida de dinero cada vez que un comensal realiza el pago de lo consumido. Teniendo en cuenta que el Equipo accede a información confidencial, el cliente desea que se firme un acuerdo de confidencialidad. Realizar las Historias de Usuario, el prototipado de pantalla, la estimación de tiempos de las Historias. Realizar el presupuesto aproximado de costo mediante un documento formal, El equipo podrá utiliza como referencia los valores publicados en el sitio: https://cpcipc.org.ar/honorarios-recomendados/ . Buscar al menos 3 Sitios Web del mercado que se dediquen al mismo rubro.



HU-01: Registrar pago de un consumo

Como cajero, quiero registrar el pago realizado por un comensal para llevar el control de los ingresos de la caja.

Criterios de aceptación:

- El sistema debe mostrar el total pendiente del consumo.
- Debe permitir seleccionar el medio de pago.
- Debe validar que el importe sea mayor que cero y no exceda el saldo pendiente.
- Al confirmar el pago, debe registrarse automáticamente un movimiento de ingreso.
- El sistema debe actualizar el saldo de caja.

HU-02: Registrar egresos de caja

Como encargado, quiero registrar los egresos de dinero para mantener actualizado el saldo disponible.

Criterios de aceptación:

- Debe permitir ingresar monto, motivo y fecha.
- El monto debe ser positivo.
- El egreso debe quedar asociado al empleado responsable.
- El saldo debe actualizarse automáticamente.
- El sistema debe impedir operaciones no autorizadas.

HU-03: Consultar movimientos

Como encargado, quiero visualizar todos los movimientos de la caja para controlar los ingresos y egresos de la sucursal.

Criterios de aceptación:

- Mostrar fecha, tipo, descripción e importe.
- Permitir filtrar por fechas y tipo de movimiento.
- Mostrar el saldo actual.
- Permitir consultar movimientos históricos.

HU-04: Apertura y cierre de caja

Como cajero, quiero abrir y cerrar la caja para controlar el dinero registrado durante cada jornada.

Criterios de aceptación:

- Permitir ingresar el saldo inicial.
- Impedir registrar operaciones con la caja cerrada.
- Mostrar un resumen de ingresos y egresos al cerrar.
- Registrar la fecha, hora y responsable del cierre.

HU-05: Administrar sucursales

Como administrador de la cadena, quiero registrar y consultar sucursales para gestionar las cajas de todos los restaurantes.

Criterios de aceptación:

- Permitir registrar nombre y dirección.
- Asociar una caja a cada sucursal.
- Impedir que una sucursal tenga dos cajas activas bajo el modelo propuesto.
- Permitir consultar los datos de cada sucursal.

HU-06: Controlar acceso al sistema

Como administrador, quiero que los empleados accedan según sus permisos para proteger la información financiera.

Criterios de aceptación:

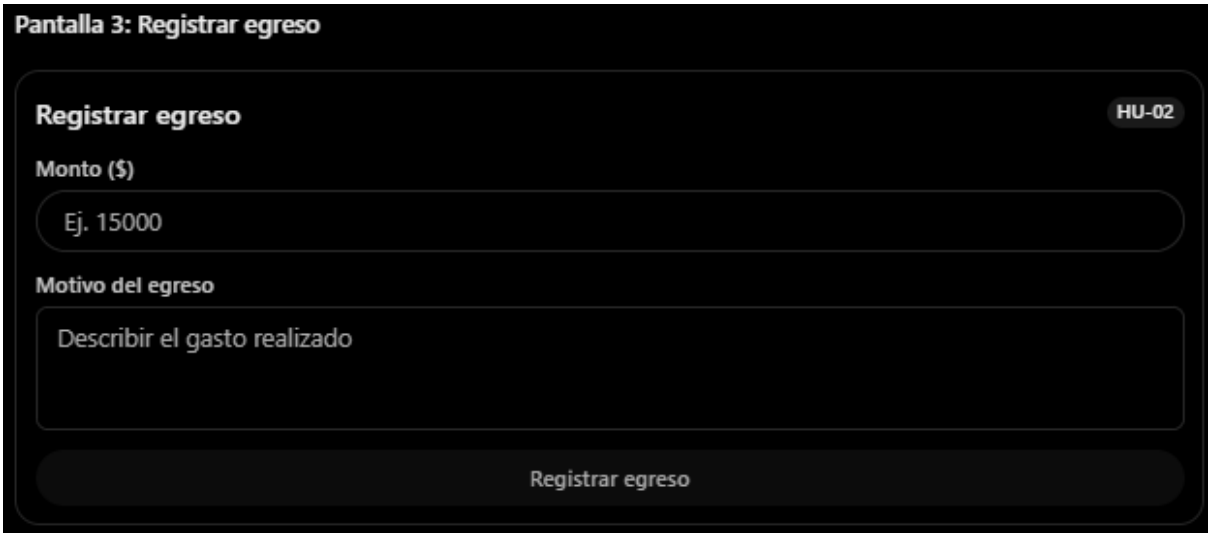
- Los usuarios deben autenticarse.
- Los cajeros pueden registrar pagos.
- Los encargados pueden registrar egresos y consultar reportes.
- Los administradores pueden consultar todas las sucursales.
- Los accesos no autorizados deben rechazarse.

Historia	Puntos	Tiempo estimado
HU-01 Registrar pagos	8	16 h
HU-02 Registrar egresos	5	10 h
HU-03 Consultar movimientos	5	10 h
HU-04 Apertura y cierre	8	16 h

HU-05 Administrar sucursales	5	10 h
HU-06 Control de acceso	8	16 h
Total	39	78 h







Presupuesto (RedSelection)+54 2613 111111

Entre: La cadena de restaurantes _____, en adelante denominada *EL CLIENTE*, y el equipo de desarrollo _____, en adelante denominado *EL EQUIPO*.

Primera – Objeto

El presente acuerdo tiene por finalidad proteger la información confidencial a la que EL EQUIPO tenga acceso durante el análisis, diseño, desarrollo y mantenimiento del sistema de gestión de cajas de las sucursales.

Segunda – Información confidencial

Se considera información confidencial:

- Los ingresos, egresos y saldos de caja.
- Los datos de pagos y consumos de los comensales.
- La información comercial y financiera de las sucursales.
- Las credenciales de acceso y configuraciones internas.
- Los documentos técnicos y cualquier otra información identificada como confidencial por EL CLIENTE.

Tercera – Obligaciones del equipo

EL EQUIPO se compromete a:

1. No divulgar información confidencial a terceros sin autorización escrita.
2. Utilizar la información únicamente para el desarrollo del sistema contratado.
3. Implementar medidas razonables de seguridad para proteger los datos.
4. Limitar el acceso a los integrantes que necesiten la información para cumplir sus funciones.
5. Comunicar inmediatamente cualquier incidente de seguridad detectado.

Cuarta – Excepciones

No se considerará confidencial la información que sea pública sin incumplimiento de este acuerdo, que ya fuera legítimamente conocida por EL EQUIPO, que se obtenga lícitamente de terceros o cuya divulgación sea legalmente obligatoria.

Quinta – Vigencia

Las obligaciones de confidencialidad permanecerán vigentes durante el proyecto y por tres años posteriores a su finalización, salvo que las partes acuerden otro plazo o que una obligación legal exija una protección mayor.

Sexta – Devolución de información

Al finalizar el proyecto, EL EQUIPO deberá devolver o eliminar la información confidencial del cliente, salvo aquella cuya conservación sea exigida por ley.

Séptima – Incumplimiento

El incumplimiento podrá dar lugar a las responsabilidades y acciones legales que correspondan conforme a la legislación aplicable.

Presupuesto (en dólares): $3.500

Lugar y fecha: _____

Firma de EL CLIENTE: _____

Firma del representante de EL EQUIPO: _____

Sitios web de ejemplo

- https://maxirest.com.ar/?utm_source=chatgpt.com
- https://www.capta.ar/?utm_source=chatgpt.com
- https://fu.do/es-ar/?utm_source=chatgpt.com

b) Investigación:
-DevOps:

> ➢ ¿Qué significa DevOps, para que sirve y cómo funciona? ¿Qué relación tiene con las metodologías Ágiles y cuales son sus diferencias? Realizar un gráfico de DevOps mostrando las herramientas que se sugieren utilizar explicando cada una de ellas.

-Ambiente de Producción y Despliegue en Producción:

> ➢ ¿Qué significa que una aplicación se despliega en un entorno de Producción? ¿Cuál es la diferencia en el desarrollo de software de un entorno de desarrollo, entorno de pruebas y entorno de producción?
> ➢¿Qué es un dominio de internet, para qué sirve y cómo funciona? Dar 5 ejemplos de dominios conocidos ¿Qué es un servidor DNS (Sistema de Nombre de Dominio)? ¿Cómo se pueden obtener un dominio en Argentina y en el resto del mundo? ¿Cómo puede obtenerse un dominio gratuito?
> ➢¿Qué es un servidor web? ¿Qué significa poder hacer Hosting de una aplicación? ¿Qué características y costos tiene DON WEB para aplicaciones de tipo SpringBoot? ¿Qué servidores web internacionales existen para realizar Hosting?
> ➢¿Qué es DOCKER y para qué sirve? Realizar me manera Gráfica de funcionamiento para una aplicación SpringBoot. ¿Qué es KUBERNET y para qué sirve? Realizar me manera Gráfica de funcionamiento para una aplicación SpringBoot. Mencionar las diferencias entre DOCKER y KUBERNET.

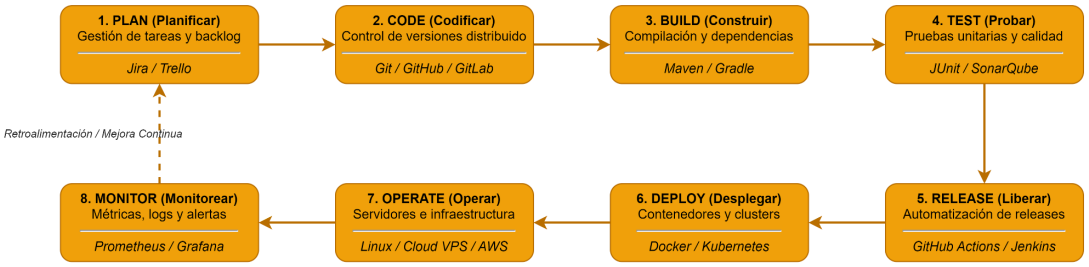
-Integración Continua y Despliegue Continuo:

➢ ¿Qué significa Integración Continua (CI) y que significa despliegue continuo (CD)?, ¿Cuáles son sus ventajas? Realizar de manera Gráfica el proceso de CI/CD en el desarrollo de una aplicación.

- **DevOps**
- Significado, utilidad y funcionamiento
    - DevOps es un acrónimo de Development (Desarrollo) y Operations (Operaciones). Representa un cambio de cultura organizacional, un conjunto de prácticas y herramientas que eliminan los silos tradicionales entre los equipos que escriben el software y los equipos responsables de desplegarlo, monitorearlo y mantener la infraestructura.
- Para qué sirve:
    - Su objetivo central es reducir el tiempo transcurrido entre la confirmación de un cambio en el código y su paso seguro a producción, elevando la frecuencia de entregas, minimizando la tasa de fallos en despliegues y recortando el tiempo medio de recuperación (MTTR) ante incidentes.
- Cómo funciona:
    - Opera sobre un ciclo continuo iterativo compuesto por fases interconectadas: Planificar (Plan) -> Codificar (Code) -> Construir (Build) -> Probar (Test) -> Empaquetar/Liberar (Release) -> Desplegar (Deploy) -> Operar (Operate) -> Monitorear (Monitor). La retroalimentación constante de la monitorización alimenta de inmediato la siguiente planificación.
- Relación y diferencias con Metodologías Ágiles
    - Relación:
        - DevOps no sustituye a las Metodologías Ágiles, sino que es su extensión natural hacia la infraestructura. Mientras que las Metodologías Ágiles resuelven la gestión de requisitos y la entrega rápida de incrementos funcionales dentro del equipo de desarrollo (Scrum/Kanban), DevOps garantiza que esos incrementos crucen la barrera técnica y operativa para llegar al usuario final sin fricciones.
    - Diferencias:
        - Enfoque y alcance principal:
            - Las metodologías Ágiles se centran en el proceso de desarrollo y la gestión de proyectos de software; su meta es planificar, gestionar requisitos cambiantes y entregar incrementos funcionales de código a través de iteraciones cortas (sprints). DevOps, en cambio, amplía el alcance más allá del desarrollo: se enfoca en la automatización, integración, despliegue, estabilidad operativa y monitoreo continuo del sistema en entornos de producción.

- Composición y límites del equipo:
    - En los marcos Ágiles los equipos están conformados principalmente por desarrolladores, analistas de negocio, Product Owners, Scrum Masters y evaluadores de calidad (QA). DevOps elimina la frontera tradicional entre desarrollo y sistemas, integrando en una misma cultura colaborativa a los ingenieros de software, administradores de sistemas, especialistas en operaciones, ingenieros de confiabilidad del sitio (SRE) y seguridad (DevSecOps).
- Concepto de software terminado (Definition of Done):
    - Para las Metodologías Ágiles, una funcionalidad se considera lista o terminada cuando el incremento de software pasa las pruebas funcionales, satisface los criterios de aceptación del usuario y queda potencialmente listo para ser desplegado. En DevOps, el software solo se considera terminado cuando está desplegado, funcionando de forma estable en el entorno de producción, monitorizado mediante telemetría y entregando valor real al usuario final.
- Rol de la automatización:
    - Aunque las metodologías Ágiles recomiendan la automatización (como pruebas unitarias o integración periódica), pueden ejecutarse exitosamente mediante ceremonias y flujos manuales de gestión. En DevOps, la automatización es un pilar indispensable: abarca desde la compilación y ejecución de pruebas hasta el aprovisionamiento de infraestructura mediante código (IaC), empaquetado en contenedores y despliegues automatizados (CI/CD).
- Objetivo de negocio y optimización:
    - El objetivo principal de Agile es optimizar la velocidad y la adaptabilidad del desarrollo frente a las necesidades del cliente y el mercado. DevOps busca optimizar la confiabilidad, la seguridad, la resiliencia operativa y minimizar el tiempo transcurrido desde que se escribe una línea de código hasta que opera de forma segura y sin interrupciones en los servidores productivos.

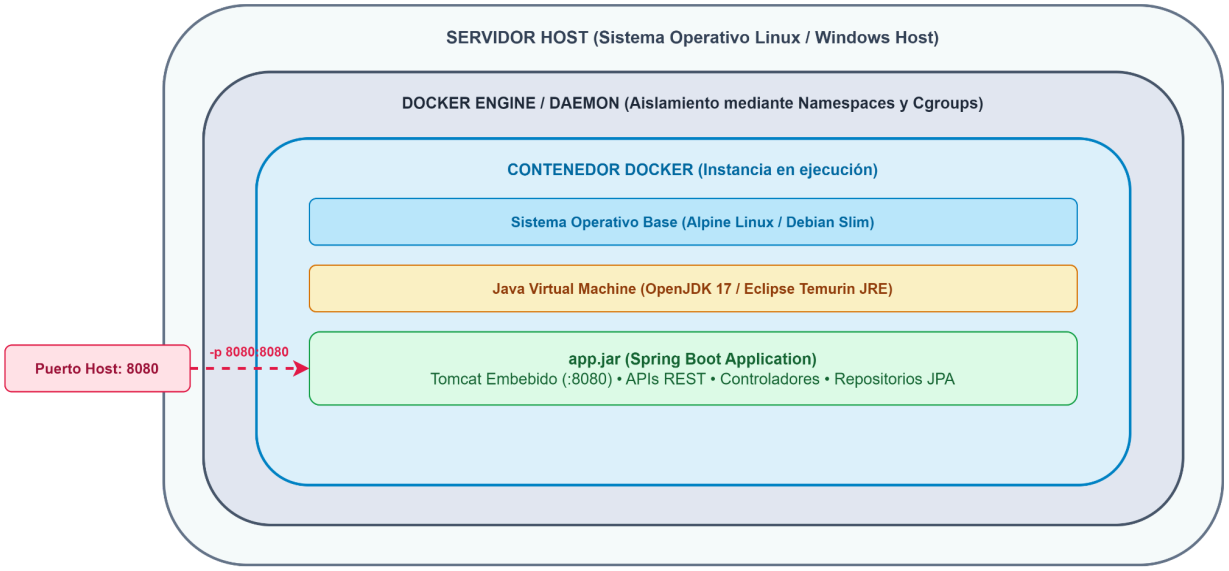
- Gráfico



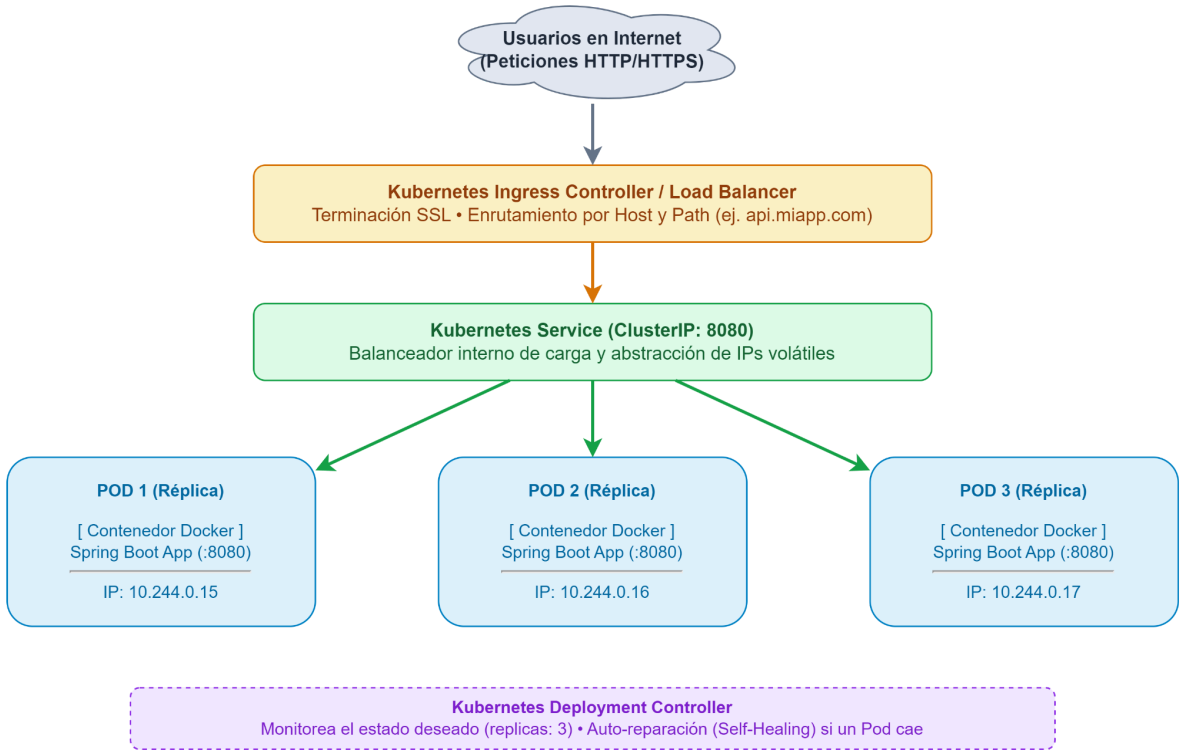
- **Ambiente de Producción y Despliegue en Producción**
  - **Despliegue en Entorno de Producción**
    - Desplegar en Producción significa poner a disposición de los usuarios finales la versión final del sistema sobre servidores reales y bases de datos transaccionales, bajo estrictos acuerdos de nivel de servicio (SLA), alta disponibilidad, auditoría y seguridad.
    - Diferencias entre Entornos
      - Entorno de Desarrollo (Dev): Espacio local o compartido donde los programadores escriben y modifican el código. Cuenta con herramientas de depuración activadas, base de datos de juguete o volcado reducido, y no requiere alta disponibilidad.
      - Entorno de Pruebas / Homologación (Staging/QA): Réplica casi idéntica de la topología de producción. Aquí el equipo de QA y los clientes validan requisitos funcionales (UAT), seguridad, integración y estrés de carga sin riesgo de corromper datos de negocio.
      - Entorno de Producción (Prod): Entorno activo donde opera el negocio en tiempo real. Los accesos están estrictamente restringidos, los certificados SSL están activos, existen mecanismos de respaldo continuo (backups) y sistemas de contingencia o balanceo de carga.
  - **Dominios de Internet y DNS**
    - Dominio de Internet: Identificador alfanumérico único asociado a una o varias direcciones IP públicas en Internet. Sirve para que los usuarios accedan a recursos web mediante nombres legibles por humanos (ej. google.com) en lugar de recordar secuencias numéricas binarias (ej. 142.250.190.46).
    - Cómo funciona: Cuando un navegador ingresa una URL, consulta a la red de servidores de nombres para resolver la correspondencia entre la cadena de texto y la dirección IP destino.
    - 5 Ejemplos de dominios conocidos:
      - google.com
      - github.com

        - mercadolibre.com.ar
        - wikipedia.org
        - uba.ar
    - Servidor DNS (Domain Name System): Sistema jerárquico y distribuido de bases de datos que actúa como la "libreta telefónica" de Internet. Recibe peticiones de resolución de nombres y devuelve los registros correspondientes (Registros A para IPv4, AAAA para IPv6, CNAME para alias, MX para servidores de correo).
    - Obtención de dominios
        - En Argentina: Se gestiona legalmente a través de la plataforma oficial NIC Argentina (nic.ar), vinculada a la cuenta de Ciudadano Digital (AFIP/MiArgentina), donde se registran extensiones territoriales como .ar y .com.ar.
        - En el resto del mundo: Se adquieren mediante empresas registradoras acreditadas por la ICANN (Internet Corporation for Assigned Names and Numbers), tales como Namecheap, GoDaddy, Cloudflare Registrar o Google Cloud Domains, para TLDs genéricos (.com, .net, .io, .org).
    - Cómo obtener un dominio gratuito: Mediante servicios de DNS dinámico o subdominios gestionados como:
        - Proveedores de subdominios gratuitos: No-IP, DuckDNS o FreeDNS.
        - Subdominios otorgados automáticamente por plataformas de hosting PaaS/Serverless (ej. mi-app.vercel.app, mi-app.github.io, mi-app.render.com).
        - Programas para estudiantes (como el GitHub Student Developer Pack, que suele incluir un dominio .me o .tech bonificado por un año).
- **Servidores Web y Hosting**
    - Servidor Web: Software o hardware configurado para aceptar solicitudes HTTP/HTTPS desde clientes (navegadores) y entregar las respuestas correspondientes. Puede servir archivos estáticos (HTML, CSS, JS, imágenes) o actuar como proxy inverso para delegar tráfico dinámico a motores de aplicación (como Tomcat, Kestrel o Node.js). Ejemplos: NGINX, Apache HTTP Server, Caddy.
    - Hosting: Servicio de provisión de infraestructura (cómputo, almacenamiento, red y memoria) en un centro de datos conectado a Internet para mantener accesible una aplicación las 24 horas del día.
    - DonWeb y aplicaciones Spring Boot:

        - Características: DonWeb es un proveedor tradicional de Argentina y Latinoamérica. Para Spring Boot, el alojamiento compartido (cPanel estándar) no suele ser compatible nativamente, ya que las aplicaciones Spring Boot corren un runtime Java independiente con su propio servidor embebido (Tomcat/Jetty). Por ende, requiere la contratación de un Servidor Cloud VPS (Virtual Private Server) o un Servidor Dedicado, donde el desarrollador tiene acceso SSH como root, instala el OpenJDK correspondiente (ej. Java 17 o 21) y gestiona la ejecución del archivo .jar como servicio de Linux (systemd) o dentro de un contenedor Docker.
        - Costos: Los planes VPS de DonWeb suelen tarifarse en moneda local (ARS) con escalones de precios según la configuración de vCPU, memoria RAM (se recomiendan mínimo 2 GB a 4 GB para una JVM de Spring Boot con MySQL) y almacenamiento SSD.
    - Servidores Web / Proveedores Internacionales para Hosting:
        - Proveedores IaaS (Infraestructura pura): Amazon Web Services (AWS EC2), Google Cloud Platform (Compute Engine), Microsoft Azure (Virtual Machines), DigitalOcean (Droplets), Linode/Akamai, Hetzner.
        - Proveedores PaaS (Plataforma gestionada directa para Java/Containers): AWS Elastic Beanstalk, Render, Railway, Fly.io, Heroku.
- **Docker y Kubernetes en Spring Boot**
    - **Docker**
        - Es una plataforma de virtualización a nivel de sistema operativo basada en contenedores. Empaqueta el código compilado de la aplicación, el runtime de Java, las librerías, dependencias y configuraciones del sistema operativo base en una unidad estándar e inmutable llamada Imagen Docker. Resuelve el problema clásico de *"en mi máquina funciona"*.



- **Kubernetes**
  - Es un orquestador de contenedores de código abierto. Sirve para automatizar el despliegue, escalado, balanceo de carga, autoreparación (*self-healing*) y actualización sin tiempo de inactividad (*zero-downtime*) de sistemas complejos basados en contenedores distribuidos en clústeres de nodos.



- **Diferencias entre Docker y Kubernetes**
  - Propósito y rol fundamental:

        - Docker es principalmente una plataforma de contenerización orientada a empaquetar una aplicación junto con sus dependencias, librerías y entorno de ejecución dentro de una imagen estandarizada e inmutable. Kubernetes, por su parte, es un orquestador de contenedores; su objetivo no es crear los contenedores, sino coordinar, desplegar, escalar y administrar clústeres donde conviven múltiples contenedores en producción.
    - Escala y alcance de ejecución:
        - Docker está diseñado para gestionar el ciclo de vida de contenedores de forma individual o en entornos locales simples sobre un único host (o máquina física/virtual). En cambio, Kubernetes opera a escala masiva sobre un clúster distribuido compuesto por múltiples nodos (servidores maestros y de trabajo), permitiendo administrar miles de instancias simultáneamente.
    - Alta disponibilidad y autoreparación (Self-Healing):
        - En Docker puro, si un contenedor falla o el proceso interno colapsa, este se detiene y requiere intervención manual o scripts externos para restablecerse (salvo políticas básicas de reinicio del motor en un solo servidor). Kubernetes monitorea de forma permanente el estado deseado frente al estado actual del sistema: si un contenedor o un nodo cae, Kubernetes destruye el contenedor averiado, levanta uno nuevo automáticamente en un nodo saludable y reconfigura las conexiones de red sin tiempos muertos.
    - Balanceo de carga y enrutamiento de red:
        - En un contenedor Docker individual, el tráfico de red se limita a la redirección básica de puertos hacia la máquina anfitriona (port mapping host-contenedor). Kubernetes provee primitivas nativas de red mucho más complejas y avanzadas (como Services tipo ClusterIP o LoadBalancer, e Ingress Controllers), las cuales distribuyen equitativamente la carga de tráfico entrante entre los diferentes contenedores réplica y

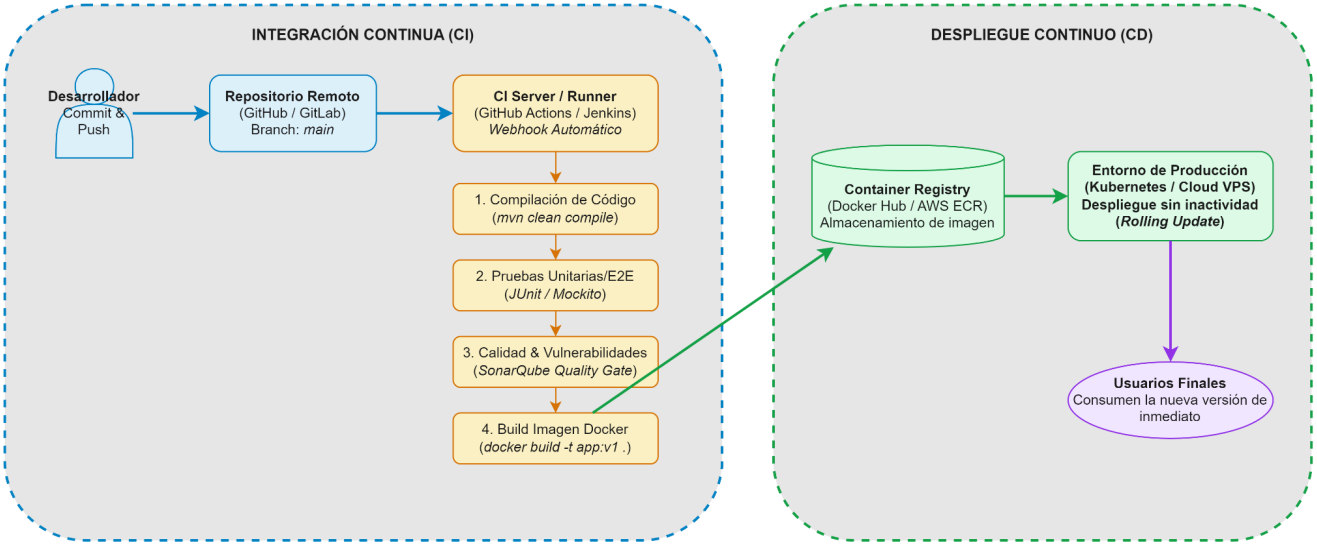
asignan una IP fija interna abstracta frente a los cambios de red.

- Escalabilidad y actualizaciones en producción:
  - Para escalar con Docker en un único servidor es necesario iniciar réplicas manuales o coordinarlas mediante herramientas complementarias. Kubernetes incluye autoescalado automático (tanto horizontal según el consumo de CPU/memoria mediante HPA como de nodos del clúster) y permite realizar actualizaciones continuas (rolling updates) y reversiones (rollbacks) progresivas sin interrumpir el servicio para los usuarios finales (zero-downtime).
- Relación de complementariedad:
  - No son tecnologías competidoras directas, sino complementarias en la cadena de valor. Docker actúa en la capa base construyendo las imágenes y corriendo el runtime del contenedor, mientras que Kubernetes utiliza dichas imágenes y contenedores para orquestarlos, gobernarlos y asegurar su estabilidad operativa en infraestructuras de gran escala.

- **Integración Continua (CI) y Despliegue Continuo (CD)**
  - **Integración Continua (CI - Continuous Integration):**
    - Práctica en la que los desarrolladores integran sus cambios de código con frecuencia en una rama principal compartida. Cada integración dispara un pipeline automatizado que compila el código y ejecuta suites de pruebas (unitarias, integración, análisis estático de seguridad).
  - **Despliegue Continuo (CD - Continuous Deployment):**
    - Práctica que extiende la Integración Continua: cada cambio que supera con éxito todos los pasos del pipeline de pruebas y análisis es liberado y desplegado de manera 100% automatizada en el entorno de producción, sin requerir aprobación o intervención humana (a diferencia de la Entrega Continua o Continuous Delivery, donde el paso a producción requiere un clic manual de confirmación).
  - **Ventajas de CI/CD**
    - Detección temprana de errores: Los defectos se identifican a los pocos minutos de ser commiteados, reduciendo drásticamente el costo de corrección.

- Reducción de riesgos en despliegues: Despliegues frecuentes y en pequeños lotes implican cambios mínimos por versión, facilitando el análisis de causa raíz y el rollback ante fallos.
- Eliminación de trabajo manual: Las tareas repetitivas de compilación, empaquetado, subida por SSH o FTP y verificación se delegan por completo a pipelines automáticos.
- Mayor velocidad de entrega (Time to Market): Las nuevas funcionalidades o parches críticos llegan al usuario en cuestión de minutos tras ser aprobados.

Proceso CI/CD (Integración y Despliegue Continuo)



c) Automatización de despliegue: Utilizando una herramienta de automatización de despliegue tipo DOCKER el equipo prepara la aplicación para despliegue en Producción. Visualizar los primeros videos de la guía “7) Docker y Deploy”.

d) Teniendo en cuenta la aplicación del Trabajo Práctico Integrador N°1 con Seguridad, deberá implementar la puesta en marcha en producción en un servidor real. Para cumplir con el objetivo utilizará los servicios de despliegue ofrecidos por la empresa de hosting “Don Web” https://donweb.com/es-ar/ . Visualizar el 3er. video de la guía “6) Docker y Deploy”.

e) Integración Continua (CI/CD):

➢ 1) Trabajo Práctico Integrador con Seguridad, para practicar el uso de servidor de integración continua, por ejemplo, para la automatización de

despliegue o test unitarios. Visualizar el 4to. video de la guía “7) Jenkins”.

➢ 2) Realizar el ejercicio que se describe en la Guía Audiovisual “7) CI/CD (Parte A)”.

➢ 3) Realizar el ejercicio que se describe en la Guía Audiovisual “7) CI/CD (Parte B)”.